



K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 4

Disk Scheduling Algorithms

Submitted by:

Name: Bhavya Rattan

Roll Number: 2401201004

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

□ Cover Section

- ♦ **Name:** Bhavya Rattan
 - ♦ **Roll Number:** 2401201004
 - ♦ **Course:** BCA (AI & DS)
 - ♦ **University:** K.R. Mangalam University, School of Engineering & Technology
 - ♦ **Subject:** Operating System Lab (ENCA252)
 - ♦ **Submitted To:** Dr. Jyoti Yadav
 - ♦ **Date:** April 25, 2026
-

□ Objective

Implement and analyze four major disk scheduling algorithms: **FCFS**, **SSTF**, **SCAN**, and **C-SCAN**. The primary goal is to minimize the **Total Seek Time** (total distance covered by the disk head) and improve I/O efficiency.

1. FCFS (First-Come, First-Served)

How it works:

Processes requests in the exact order they arrive in the disk queue.

Logic:

Simple and fair, but doesn't optimize head movement at all. Each request is served regardless of how far the disk head must travel, which can lead to wild swings across the disk surface and high cumulative seek distances.

Step-by-Step Trace (Head: 53)

Step	From	To	Seek Distance
1	53	98	45
2	98	183	85
3	183	37	146
4	37	122	85
5	122	14	108
6	14	124	110
7	124	65	59
8	65	67	2

Total Seek Time: 640

Code Output

```
bhavya@dell:~/Documents/os/os_assignment_3$ cd /home/bhavya/Documents/os/os_assignment_4
bhavya@dell:~/Documents/os/os_assignment_4$ nano disk_scheduling.py
bhavya@dell:~/Documents/os/os_assignment_4$ python3 disk_scheduling.py
```

```
=====
DISK SCHEDULING ALGORITHM SIMULATOR
=====
```

```
Enter disk request queue (space-separated): 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size (total cylinders, e.g. 200): 200
```

```
Request Queue      : [98, 183, 37, 122, 14, 124, 65, 67]
Initial Head       : 53
Disk Size          : 200 cylinders
```

```
-----
=====
RUNNING ALL ALGORITHMS
=====
```

FCFS (First Come First Serve)

```
Movement Sequence : 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
Total Seek Time    : 640 cylinders
```

2. SSTF (Shortest Seek Time First)

How it works:

Selects the request with the minimum seek distance from the current head position at each step.

Logic:

A greedy approach that always picks the nearest unvisited track. This dramatically reduces total seek time compared to FCFS, but can cause **starvation** for requests that are consistently far from the head whenever closer requests keep arriving.

Step-by-Step Trace (Head: 53)

Step	From	To	Seek Distance
1	53	65	12
2	65	67	2
3	67	37	30
4	37	14	23
5	14	98	84
6	98	122	24
7	122	124	2
8	124	183	59

Total Seek Time: 236

Code Output

```
SSTF (Shortest Seek Time First)
```

```
Movement Sequence : 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
```

```
Total Seek Time   : 236 cylinders
```

3. SCAN (Elevator Algorithm)

How it works:

The head moves in one direction (upward), servicing all requests along the way. Once it reaches the end of the disk (track 199), it reverses direction and services remaining requests on the way back.

Logic:

Provides more uniform wait times than SSTF and completely avoids starvation. Every request is guaranteed to be served within two full sweeps of the disk, making it suitable for high-load systems.

Step-by-Step Trace (Head: 53, Direction: Up)

Step	From	To	Seek Distance
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	199	16
8	199	37	162
9	37	14	23

Total Seek Time: 331

Code Output

SCAN (Elevator Algorithm)

Movement Sequence : 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14

Total Seek Time : 331 cylinders

4. C-SCAN (Circular SCAN)

How it works:

Similar to SCAN, but when the head reaches the end of the disk, it immediately jumps back to track 0 without servicing any requests on the return trip, then resumes servicing in the upward direction.

Logic:

Treats the disk tracks as a circular list, which provides even more uniform waiting times than SCAN. Requests near the beginning of the disk are never starved by the long reverse sweep. The jump from end to start counts as head movement.

Step-by-Step Trace (Head: 53, Direction: Up)

Step	From	To	Seek Distance
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	199	16
8	199	0	199
9	0	14	14
10	14	37	23

Total Seek Time: 382

Code Output

C-SCAN (Circular SCAN)

Movement Sequence : 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37
Total Seek Time : 382 cylinders

Performance Comparison

Algorithm	Total Seek Time	Efficiency Rank
SSTF	236	1st (Best)
SCAN	331	2nd
C-SCAN	382	3rd
FCFS	640	4th (Worst)

Code Output

```
=====
  PERFORMANCE COMPARISON SUMMARY
=====
Algorithm      Total Seek Time    Performance
-----
FCFS              640 cylinders
SSTF              236 cylinders ← BEST
SCAN              331 cylinders
C-SCAN            382 cylinders
```

Conclusion

- ♦ **Best Performer: SSTF** achieved the lowest total seek time (236) for this request set. It is highly efficient when requests are localized.
- ♦ **Worst Performer: FCFS** was the least efficient (640), blindly following arrival order with no regard for head position, causing excessive back-and-forth movement.
- ♦ **Observations:** While SSTF is the fastest numerically, **SCAN** and **C-SCAN** are preferred in production systems due to better fairness and guaranteed starvation prevention. C-SCAN's circular approach offers the most uniform waiting time across all disk tracks.

CODE OUTPUT:-

```
=====
  RESULT ANALYSIS
=====
Best Algorithm : SSTF (236 cylinders)
Worst Algorithm : FCFS (640 cylinders)

Conclusions
-----
• FCFS    → Simplest algorithm. Services requests in arrival
            order. Easy to implement but causes high seek time
            due to random head movement (no optimization).

• SSTF    → Always picks the closest request. Gives the lowest
            or near-lowest seek time but can starve far requests
            (they keep getting skipped). Not fair.

• SCAN    → Head sweeps back and forth like an elevator. Good
            balance of performance and fairness. No starvation.
            Used widely in real operating systems.

• C-SCAN  → Only moves in one direction, then jumps to start.
            More uniform wait time than SCAN. Fair to all
            requests. Slightly higher seek time due to the jump.

bhavya@dell:~/Documents/os/os_assignment_4$
```